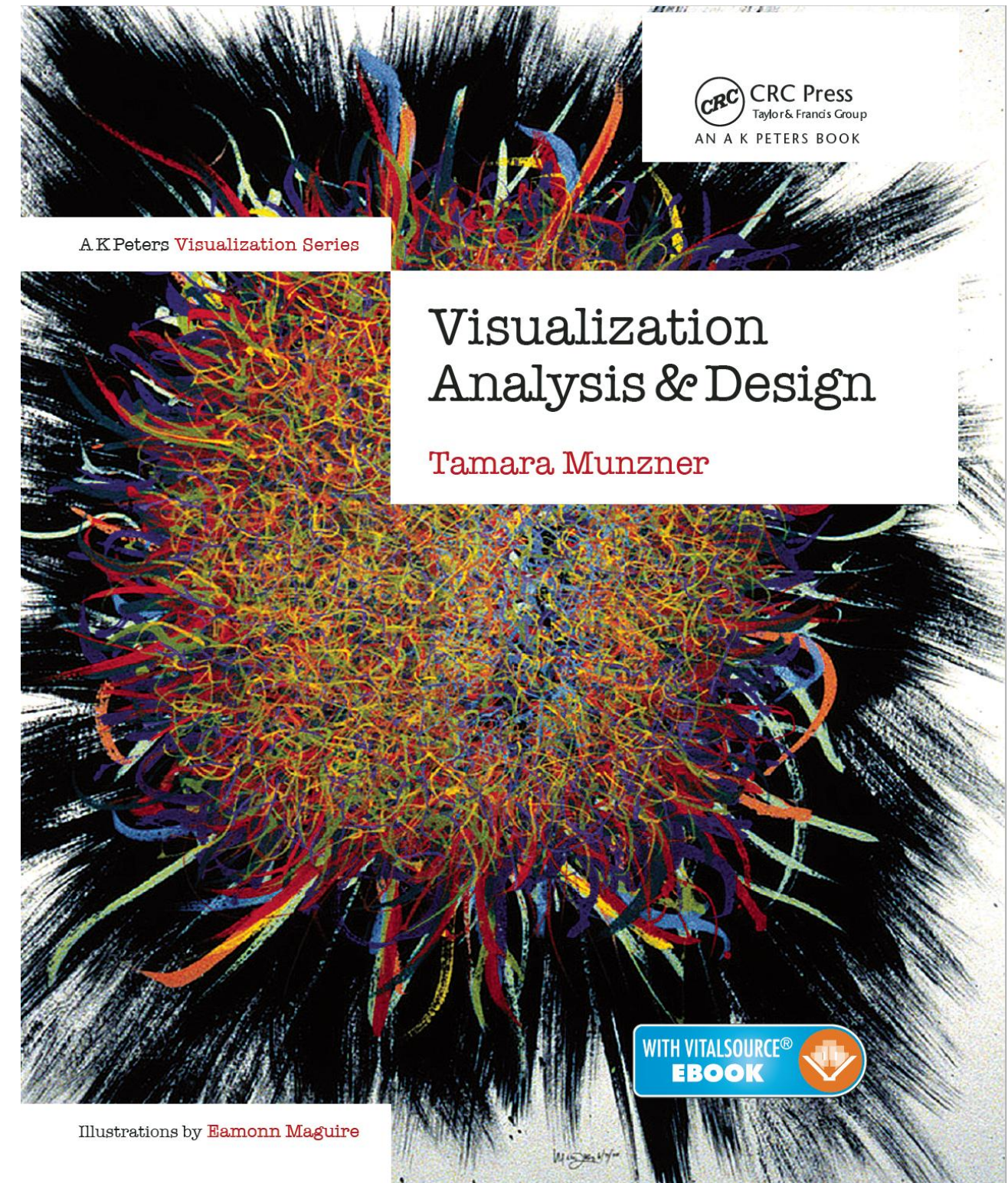


CDS505

Data Visualization & Visual Analytics

Networks and Trees

*Nur Intan Raihana Ruhaiyem, PhD.
School of Computer Sciences
Universiti Sains Malaysia*



- ✓ Networks & Trees - Introduction
- ✓ Connection: Link Marks
- ✓ Matrix Views
- ✓ Connection vs. Matrix Views - comparison
- ✓ Containment: Hierarchy Marks

DEFINE NETWORKS

Networks are defined by two things: **nodes** and **links**.

Nodes: a collection of entities which have properties that are somehow related to each other

→ E.g., people, forks in rivers, proteins, web pages, organisms, etc.

Edges/Links: connections between nodes

→ Links may be **directed** or **undirected**

→ Links may be **binary** or **weighted**

EXAMPLES

- Tournaments
- Organization charts
- Genealogy
- Diagramming (e.g., Visio)
- Biological interactions (genes, proteins)
- Computer networks
- Social networks
- Simulation and modelling
- Integrated circuit design
- River systems

NETWORK VALUES

NODES

- Usually contains the values
- Friends attributes (name, age, gender, etc.) in a social network

LINKS

- Can also have attributes
- When the friendship was established
- What type of relationship
- How many friends in common do they have

HOW TO STORE NETWORK DATA

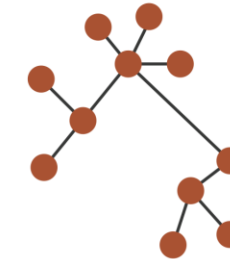
- Node and link files
- Adjacency matrix
- Nested: XML/JSON
- Many others

- The **node-link diagram** - uses the **connection channel**, where **marks** represent **links** rather than **nodes**.

➔ **Node-Link Diagrams**
Connection Marks

✓ NETWORKS

✓ TREES

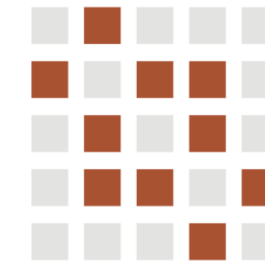


- **Adjacency matrix views** - directly show **adjacency** relationships.

➔ **Adjacency Matrix**
Derived Table

✓ NETWORKS

✓ TREES



- **Tree structure** can be shown with the **containment** channel, where enclosing link marks show hierarchical relationships through **nesting**.

➔ **Enclosure**
Containment Marks

✗ NETWORKS

✓ TREES



HOW TO CREATE NETWORK DATA

1. Group by **common attribute**.
2. Identify **nodes**, extract **links**.

→ Social network data extract

- *Loop through all of the messages.*
- *Add to a list of all users.*
- *Add to an edge list that has all "mentions" of another user.*

→ E.g., Co-Authorship Networks

- <https://observablehq.com/@john-guerra/uist-2020-co-authorship-network>

Idiom: Radial Node-Link Tree

- **Data - Tree**
- **Encoding**
 - link connection marks
 - point node marks
 - radial axis orientation
 - angular proximity: siblings
 - distance from center: depth in tree
- **Tasks**
 - understanding topology, following paths
- **Scalability**
 - 1K - 10K nodes

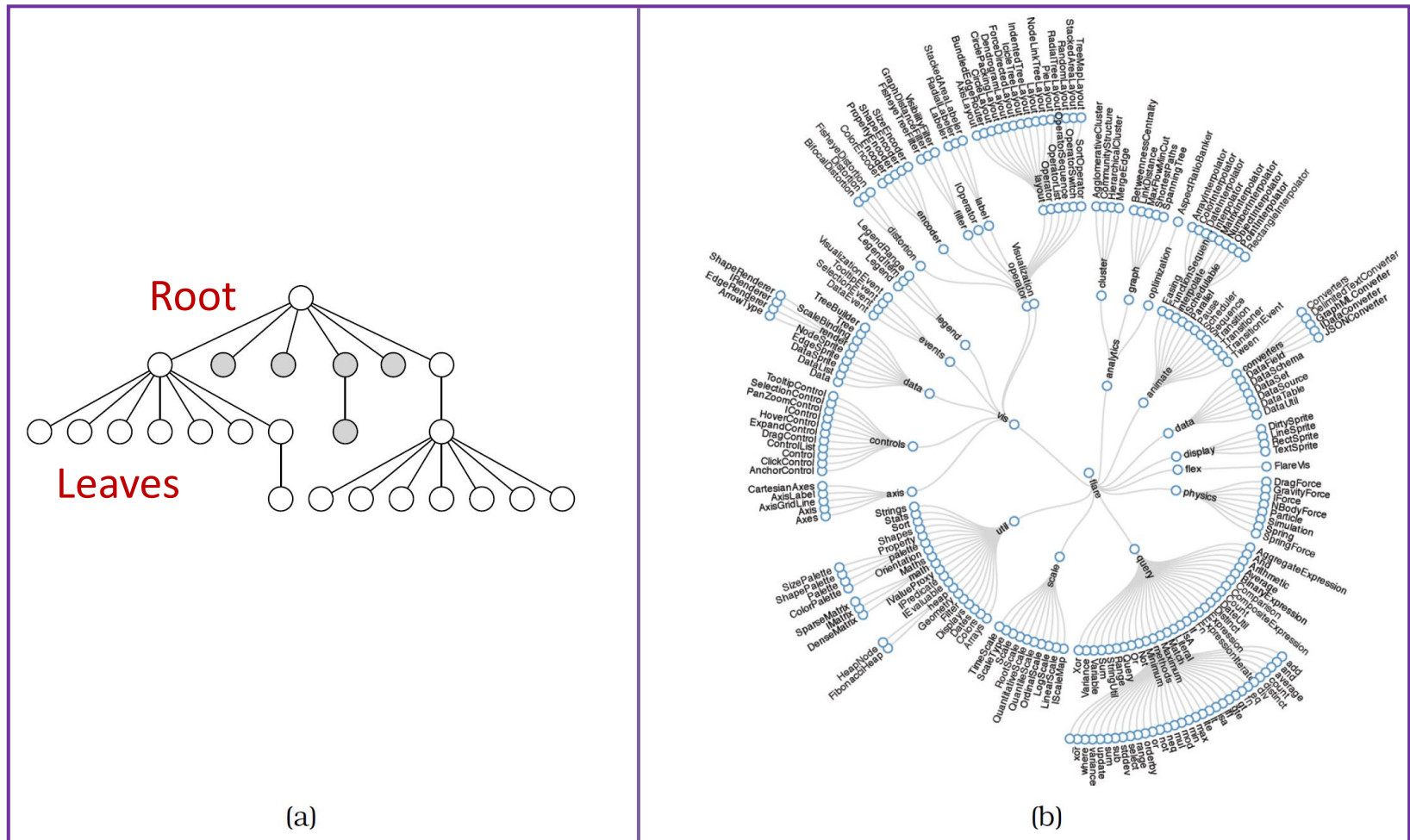


Figure 9.2: Node-link layouts of small trees.

(a) Triangular vertical for tiny tree. From [Buchheimetal.02].

(b) Spline radial layout for small tree. From <http://mbostock.github.com/d3/ex/tree.html>.

Two layouts of a 5161-node tree

- **Rectangular horizontal node-link layout**
 - The edges are coloured with a purple to orange continuous colourmap
- **BubbleTree node-link layout**
 - A radial approach - subtrees are laid out in full circles rather than partial circular arcs.
 - Spatial position does encode information about tree depth, but as relative distances to the centre of the parent rather than as absolute distances in screen space.

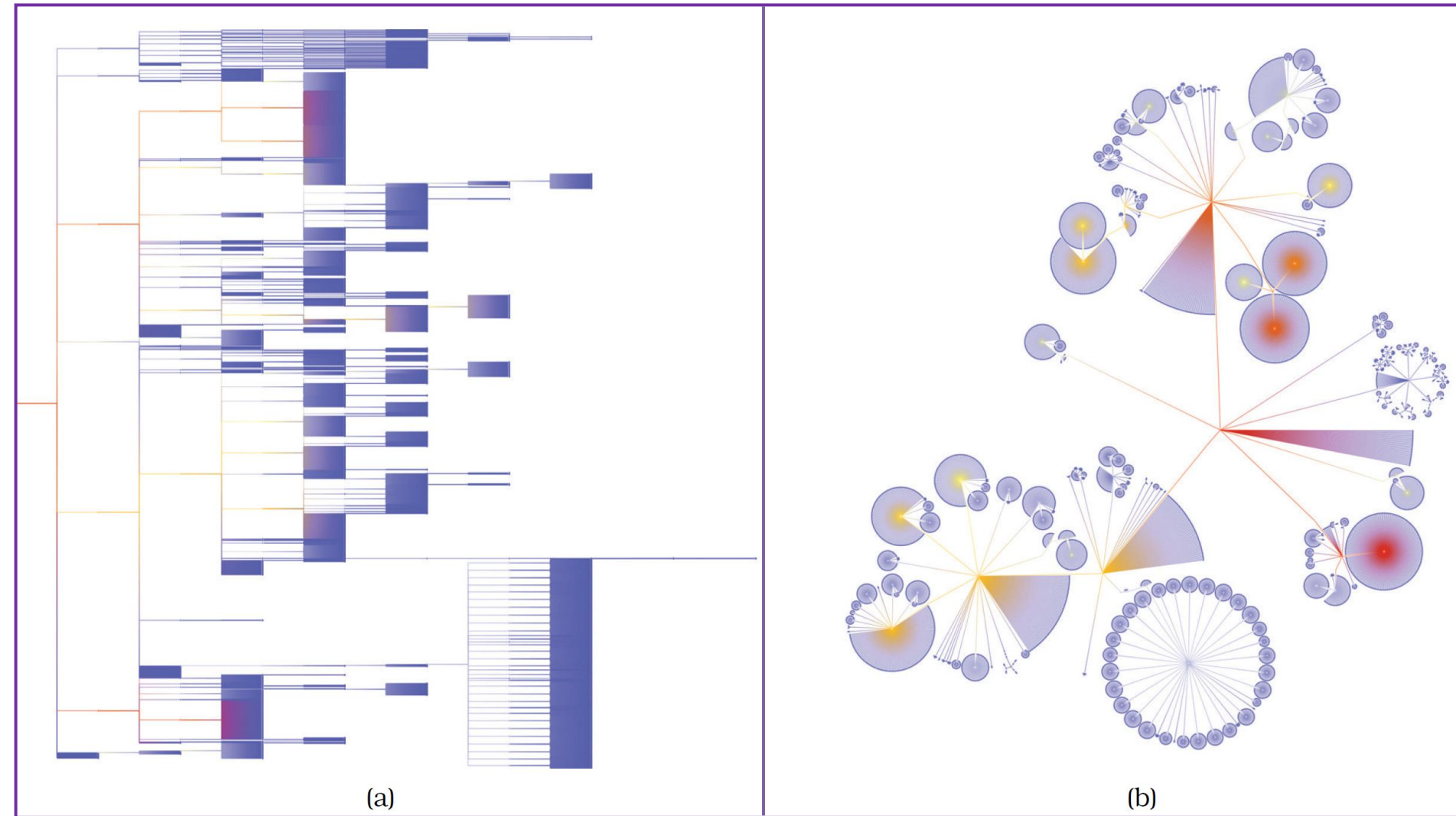


Figure 9.3: Two layouts of a 5161-node tree.

(a) Rectangular horizontal node-link layout.

(b) BubbleTree node-link layout [Grivet et al. 06].

- Directly connected by a **single link** - the **tightest** grouping
- A **long path** of multiple hops → **less closely** grouped.
- The # of hops within a path:- the # of individual links that must be traversed to get from one node to another **is** a **network-oriented way to measure distances**.
- Well suited for tasks - understanding the **network topology**: the direct and indirect connections between nodes in terms of the # of hops, e.g.,
 - *finding **all possible paths** from one node to another*
 - *finding the **shortest path** between two nodes*
 - *finding **all the adjacent nodes** one hop away from a target node*
 - *finding nodes that act as a **bridge** between two components of the network that would otherwise be disconnected*

Idiom: Force-Directed Placement

- **Visual encoding**
 - link connection marks, node point marks
- **Considerations**
 - spatial position: no meaning directly encoded
 - left free to minimize crossings
 - proximity semantics?
 - sometimes meaningful
 - sometimes arbitrary, artifact of layout algorithm
 - tension with length
 - long edges more visually salient than short
- **Tasks**
 - explore topology; locate paths, clusters
- **Scalability**
 - node/edge density $E < 4N$: hairball of visual clutter (see Figure 9.5)

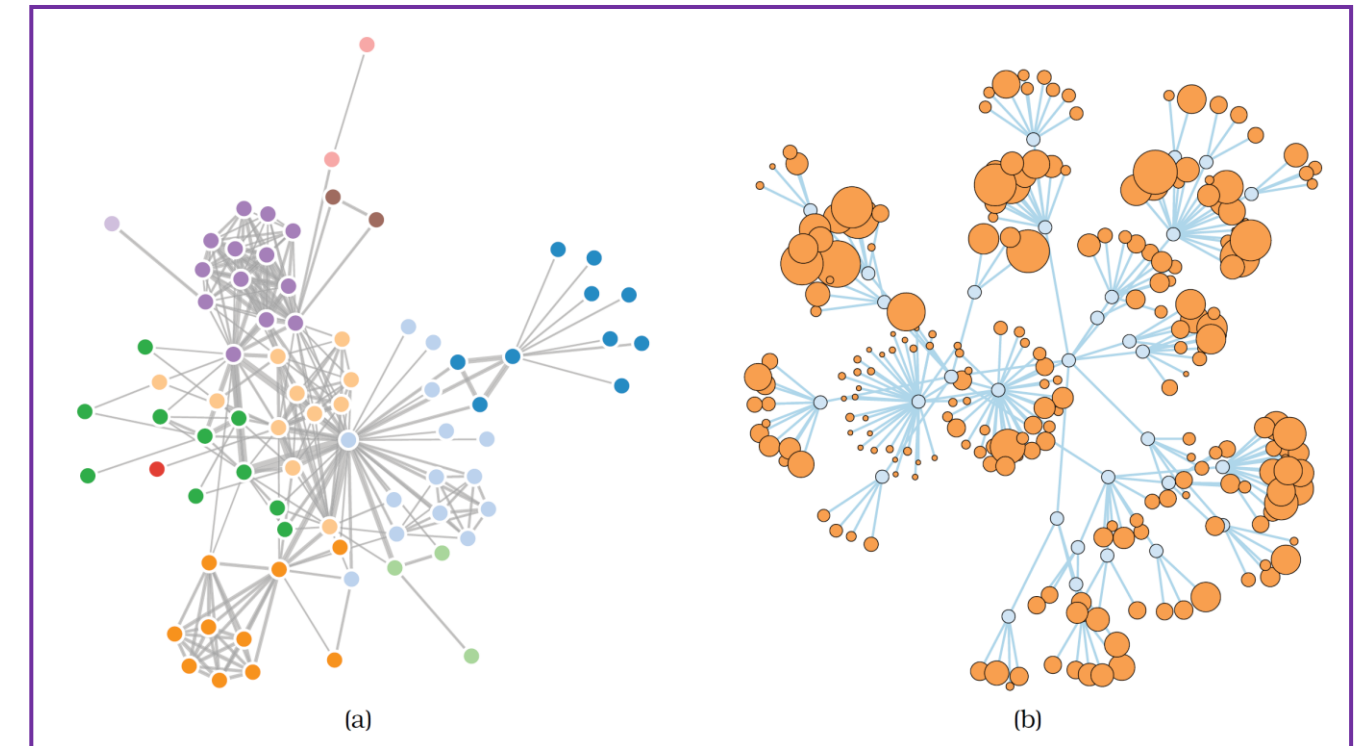


Figure 9.4: Node-link layouts of small networks.

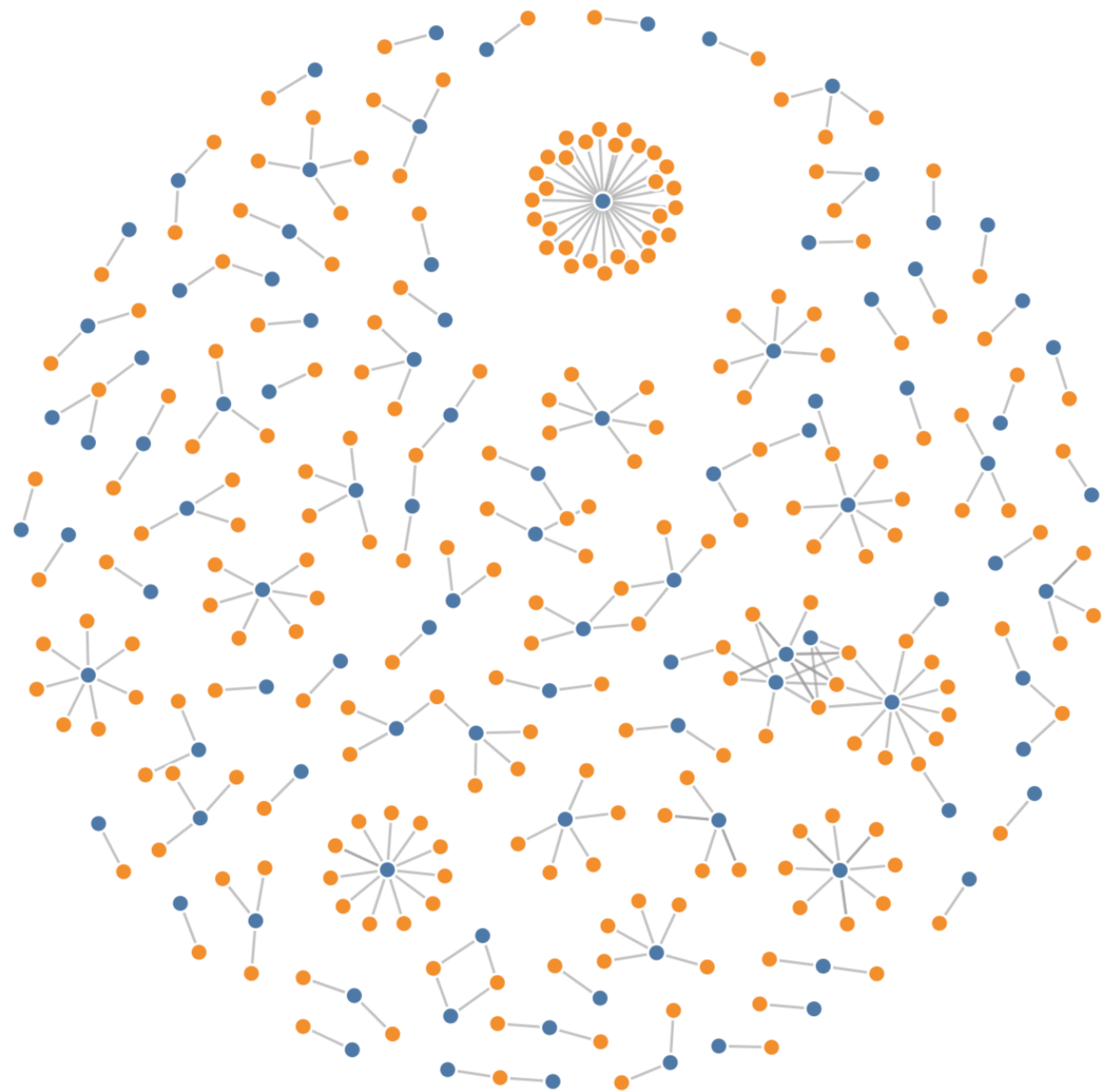
- (a) Force-directed placement of small network of 75 nodes, with size coding for link attributes.
- (b) Larger network, with size coding for node attributes. From <http://bl.ocks.org/mbostock/4062045> and <http://bl.ocks.org/1062288>.

Other example – disjoint FDP

<https://observablehq.com/@d3/disjoint-force-directed-graph>

Connection: Link Marks – Disjoint Force-Directed

■ Cited Works ■ Citing Patents



Using '***positioning forces***' (push nodes towards a desired position) instead of '***centering force***' (translate nodes uniformly).

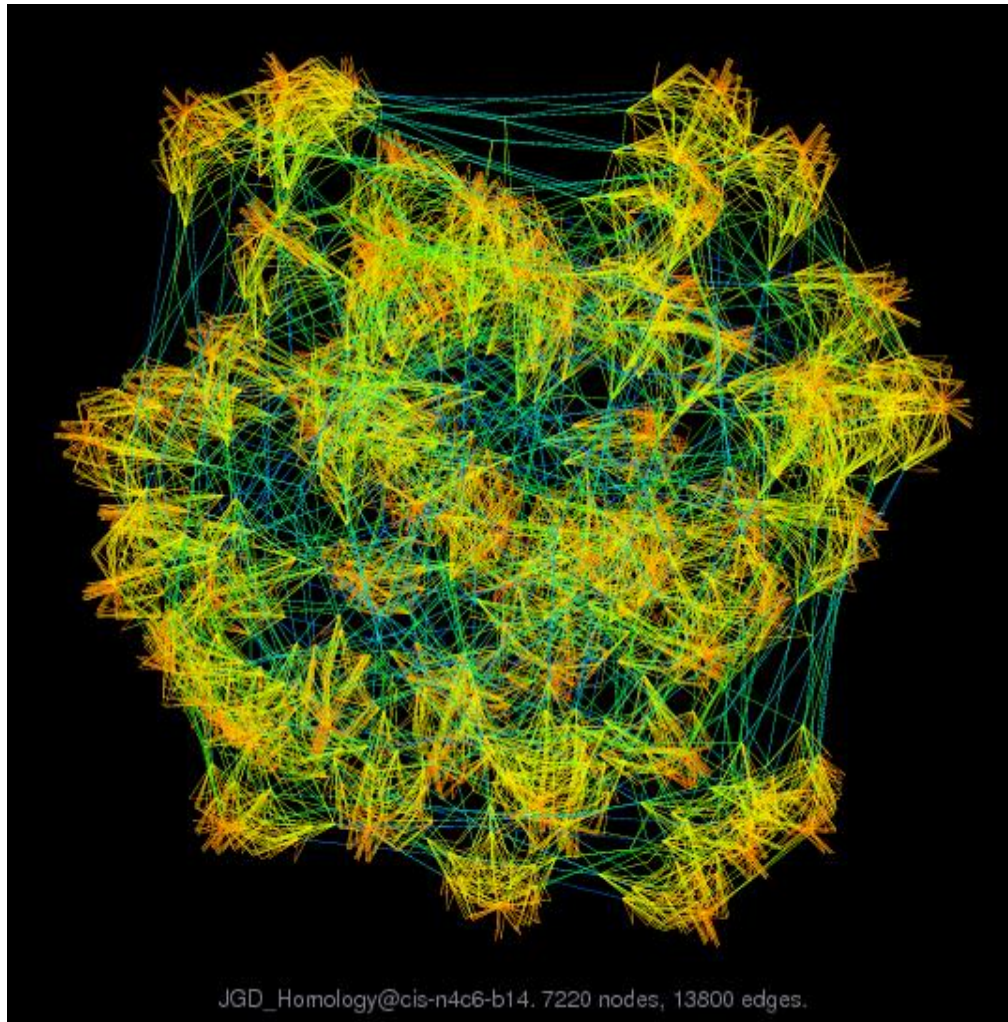
<https://observablehq.com/@d3/disjoint-force-directed-graph>

- In one variant, the network elements are positioned according to a **simulation of physical forces** where nodes push away from each other while links act like springs, start by placing nodes randomly within a spatial region and then iteratively refine their location according to the pushing and pulling of these simulated forces to gradually improve the layout.
- Weakness - the layouts are often nondeterministic → that they will look different each time the algorithm is run. **Region-based identifications** such as “the stuff in the upper left corner” might **change** between runs.
- Major weakness - scalability, both in terms of the **visual complexity** of the layout and the **time** required to compute it.

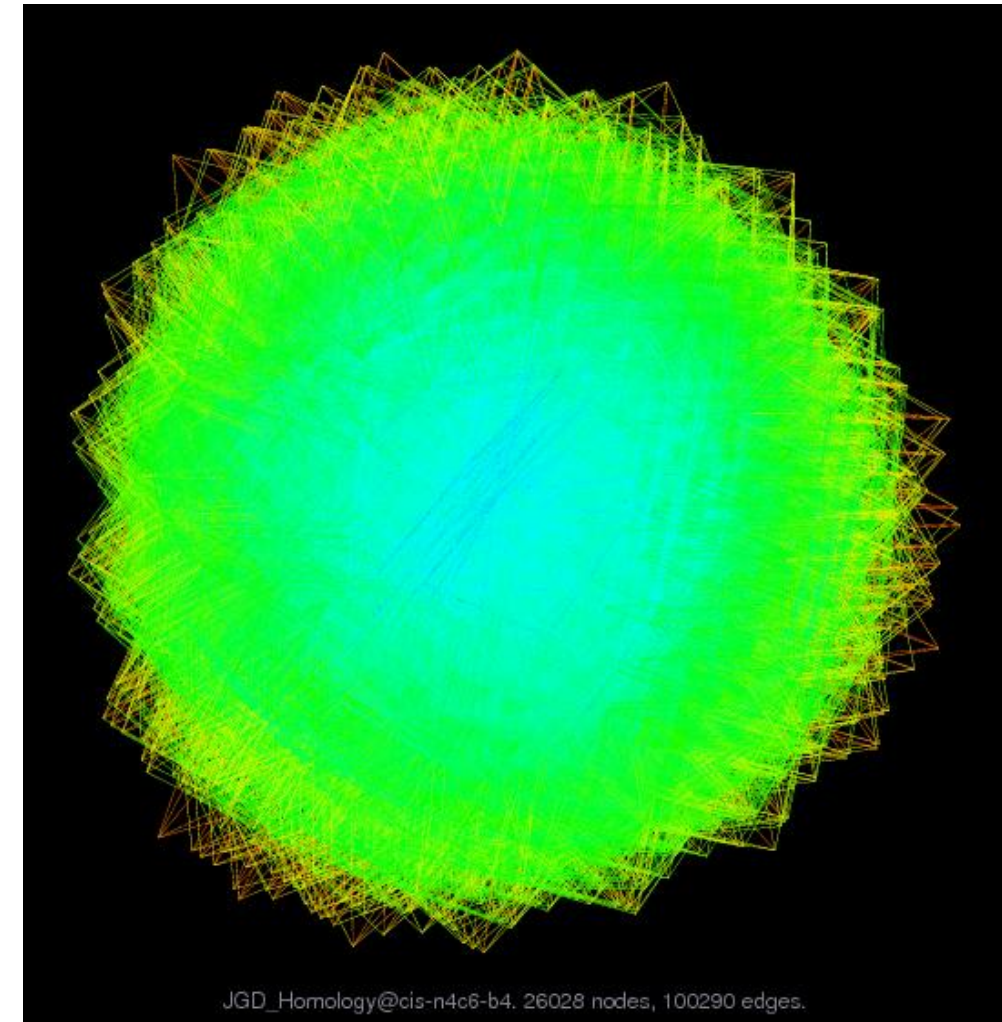
Idiom: SFDP (multi-level Scalable Force-Directed Placement)

- **Data**
 - original: network
 - derived: cluster hierarchy atop it, e.g., by coarsening the original network into successively simpler networks
- **Considerations**
 - fundamental use of space
 - speed/quality of layout
- **Scalability**
 - nodes, edges: 1K-10K
 - hairball problem eventually hits

Connection: Link Marks – SFDP



(a)



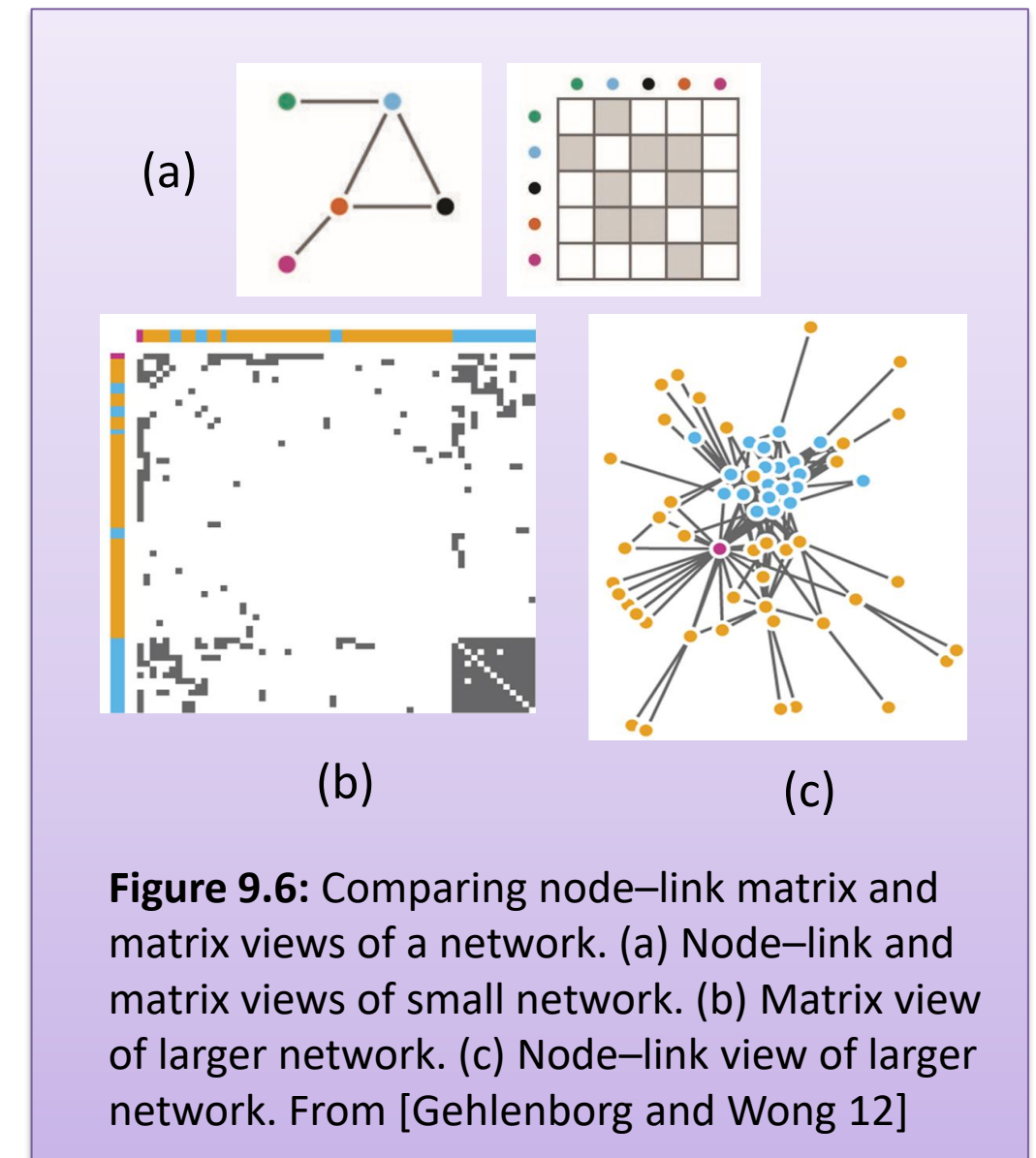
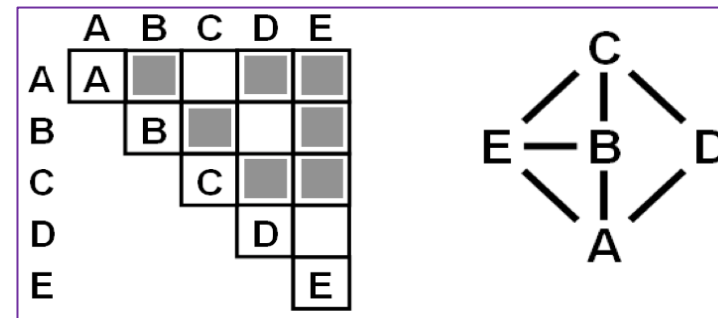
(b)

Figure 9.5: Multilevel graph drawing with sfdp [Hu 05].

- (a) Cluster structure is visible for a large network of 7220 nodes and 13,800 edges (coloured by length). The dense clusters with short orange and yellow edges can be distinguished from the long blue and green edges between them
- (b) A huge graph of 26,028 nodes and 100,290 edges is a “hairball” without much visible structure. From [Hu 14].

Adjacency Matrix View

- **Data:** network
 - transform into same data/encoding as heatmap
- **Derived data:** table from network
 - 1 quantitative attribute
 - weighted edge between nodes
 - 2 categorical attributes: node list x 2
- **Visual encoding**
 - cell shows presence/absence of edge
- **Task**
 - Identify clusters (topology)
 - Summarize topology/distribution
- **Scalability**
 - 1K nodes, 1M edges



Connection vs. Matrix Views Comparison

- **Node-link diagram strengths**
 - topology understanding, path tracing
 - intuitive, no training needed
- **Adjacency matrix strengths**
 - predictability, scalability, supports reordering
 - some topology tasks trainable
- Empirical study
 - node-link best for small networks
 - matrix best for large networks
 - *if tasks don't involve topological structure!*

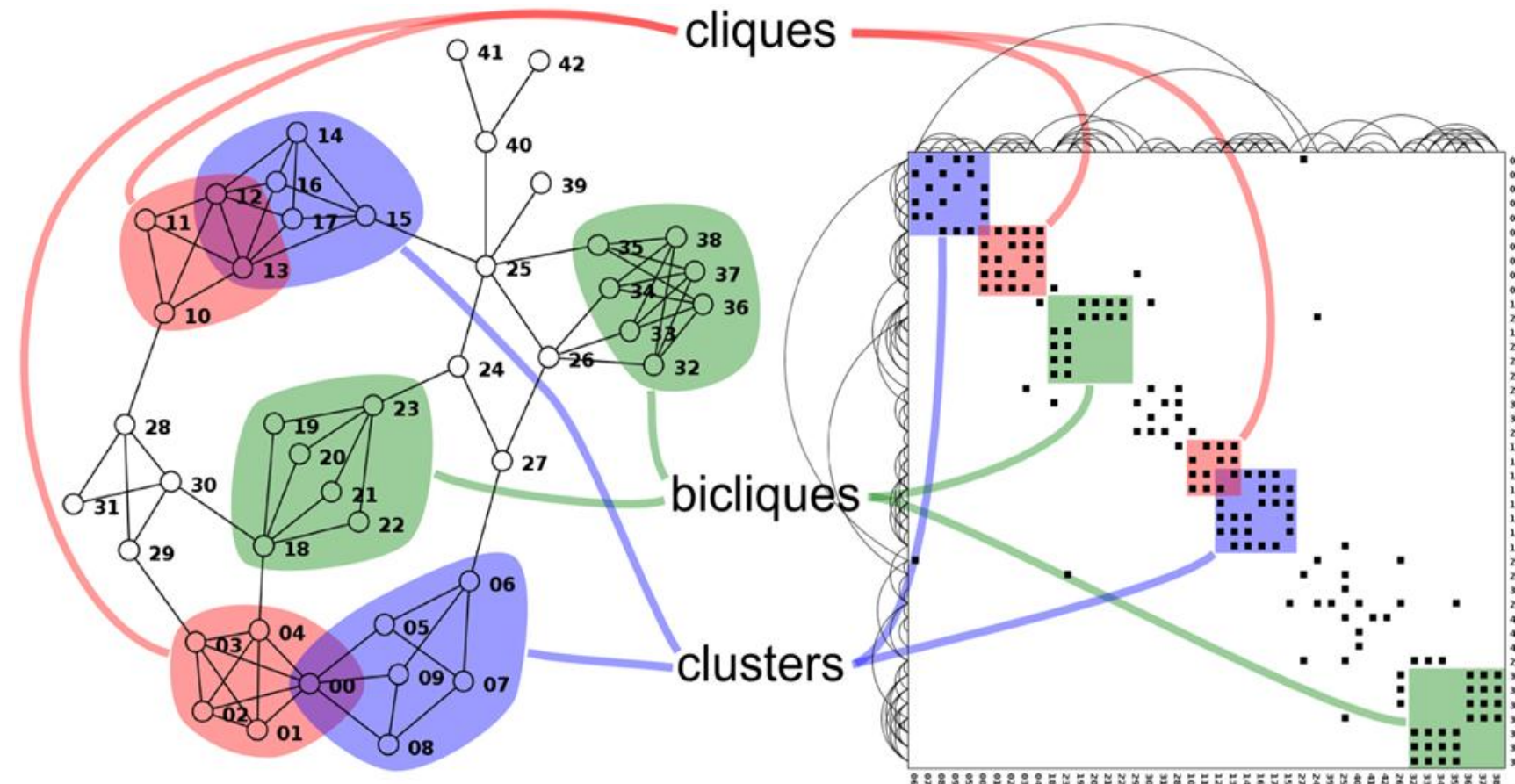


Figure 9.7: Characteristic patterns in matrix views and node-link views: both can show cliques and clusters clearly. From [McGuffin 12, Figure 6].

Idiom: Treemap

- **Data:** tree
 - 1 quantitative attribute at leaf nodes
- **Encoding**
 - area containment marks for hierarchical structure
 - rectilinear orientation
 - size encodes quantitative attribute
- **Tasks**
 - Query attribute at leaf nodes
- **Scalability**
 - 1M leaf nodes

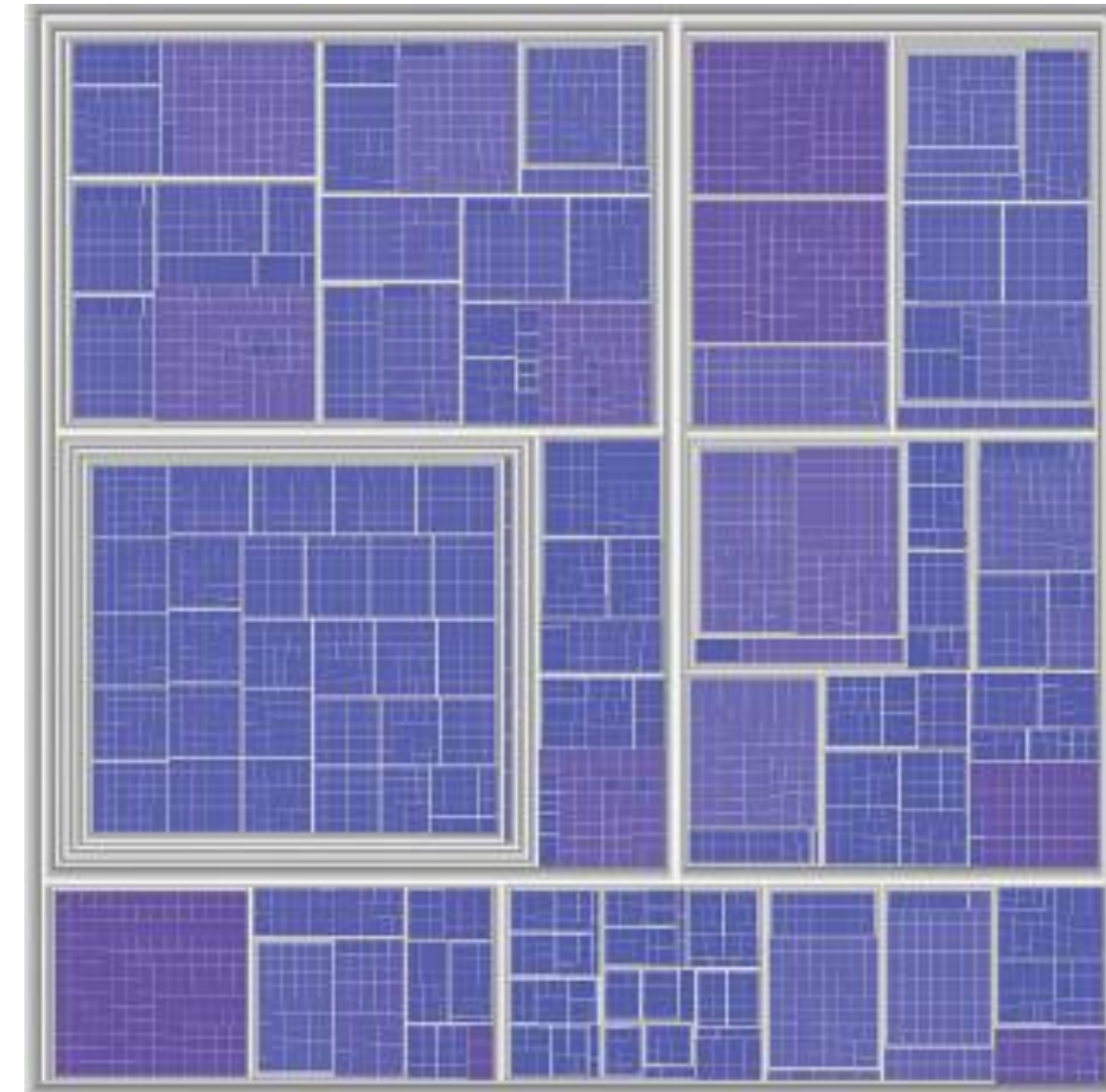


Figure 9.8: Treemap layout showing hierarchical structure with containment rather than connection, in contrast to the node–link diagrams of the same 5161-node tree. All of the children of a tree node are enclosed within the area allocated that node, creating a nested layout

Containment: Hierarchy Marks

Tree drawing idioms comparison

- **Data shown**

- link relationships
- tree depth
- sibling order

- **Design choices**

- connection vs containment link marks
- rectilinear vs radial layout
- spatial position channels

- **Considerations**

- arbitrary?
- information density?
 - avoid wasting space

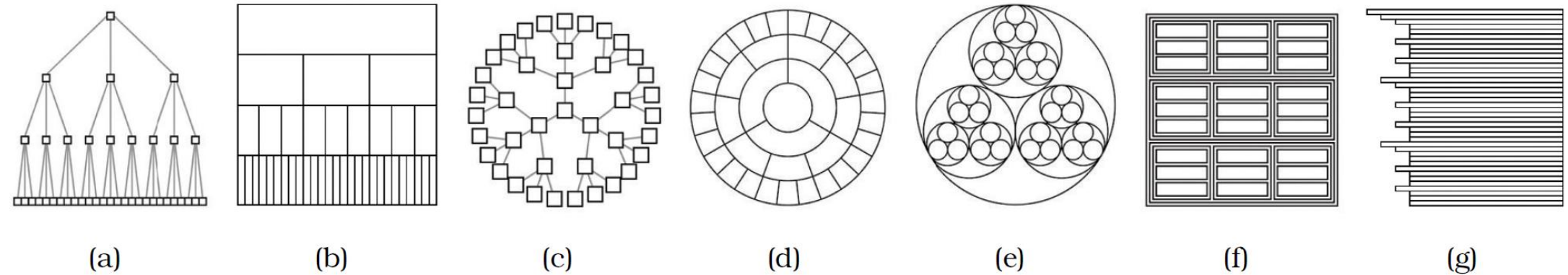


Figure 9.9: Seven visual encoding idioms showing the same tree dataset, using different combinations of visual channels.

- (a) Rectilinear vertical node-link:** link relationships – connection, tree depth - vertical spatial position, sibling order - horizontal spatial position
- (b) Icicle:** tree depth - vertical spatial position and size, link relationship and sibling order - horizontal spatial position
- (c) Radial node-link:** link relationships – connection, tree depth-radial depth spatial position, sibling order - radial angular position.
- (d) Concentric circles:** tree depth - radial depth spatial position and size, link relationships and sibling order - radial angular spatial position
- (e) Nested circles:** radial containment - nesting level, tree depth-size .
- (f) Treemap:** rectilinear containment - nesting level, tree depth-size
- (g) Indented outline:** tree depth - horizontal spatial position, sibling order - link relationships and vertical spatial position. From [McGuffin and Robert 10].

System: GrouseFlocks

- Data: compound graphs
 - network
 - cluster hierarchy atop it
 - derived or interactively chosen
- Visual encoding
 - connection marks for network links
 - containment marks for hierarchy
 - point marks for nodes
- Dynamic interaction
 - select individual metanodes in hierarchy to expand/contract

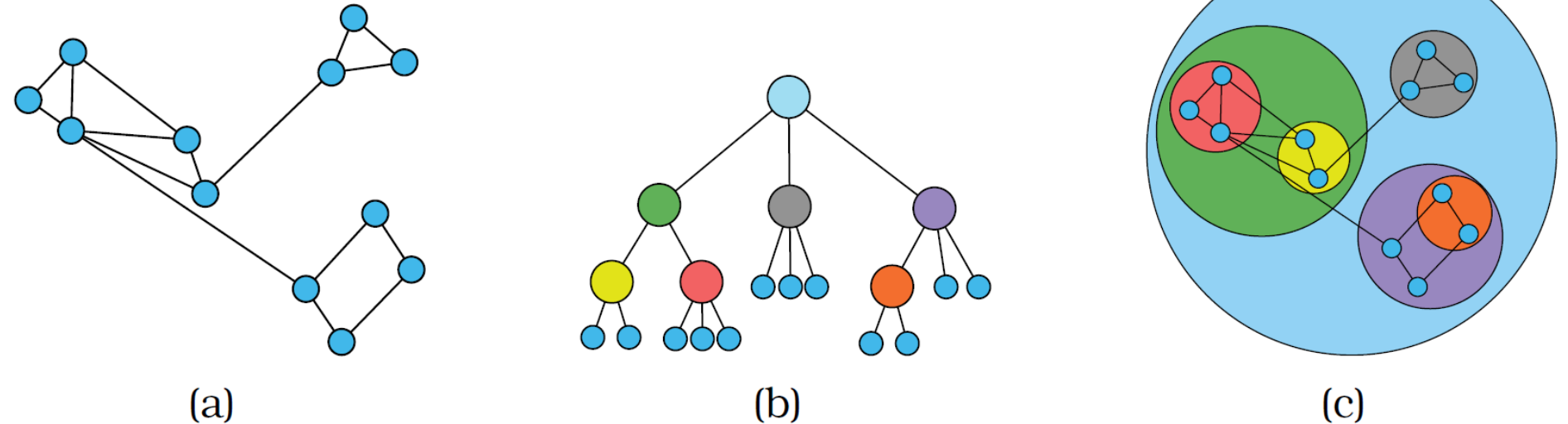


Figure 9.10: GrouseFlocks uses containment to show graph hierarchy structure.

- (a) Original graph.
- (b) Cluster hierarchy built atop the graph, shown with a node–link layout. The nodes in the network are the leaves of the tree. Thus, the interior nodes of the tree encompass multiple network nodes.
- (c) Network encoded using connection, with hierarchy encoded using containment.

From [Archambault et al. 08].

- Visualization Analysis and Design. Munzner. AK Peters / CRC Press, Oct 2014 - Chap 9: Arrange Networks and Trees
- Visual Analysis of Large Graphs: State-of-the-Art and Future Research Challenges. von Landesberger et al. Computer Graphics Forum 30:6 (2011), 1719–1749.
- Simple Algorithms for Network Visualization: A Tutorial. McGuffin. Tsinghua Science and Technology (Special Issue on Visualization and Computer Graphics) 17:4 (2012), 383–398.
- Drawing on Physical Analogies. Brandes. In Drawing Graphs: Methods and Models, LNCS Tutorial, 2025, edited by M. Kaufmann and D. Wagner, LNCS Tutorial, 2025, pp. 71–86. Springer-Verlag, 2001.
- Treevis.net: A Tree Visualization Reference. Schulz. IEEE Computer Graphics and Applications 31:6 (2011), 11–15. <http://www.treevis.net>
- Perceptual Guidelines for Creating Rectangular Treemaps. Kong, Heer, and Agrawala. IEEE Trans. Visualization and Computer Graphics (Proc. InfoVis) 16:6 (2010), 990–998.